

Research on Google Agent Development Kit (ADK) and Designing an Analytics Sub-agent

1. Overview of Google ADK

What is ADK?

Google's **Agent Development Kit (ADK)** is a flexible, modular framework for developing and deploying AI agents. According to the ADK documentation, the framework is *model-agnostic* and *deployment-agnostic*, designed to be compatible with different frameworks and to make agent development feel like software development ¹. ADK was announced at Google Cloud NEXT 2025, where the Google Developers blog described it as an open-source framework powering multi-agent systems within Google products such as Agentspace and the Customer Engagement Suite ². The blog identifies ADK's **core pillars**—*Build, Interact, Evaluate* and *Deploy*—which provide capabilities across the agent development lifecycle ³.

Recent enhancements (May 2025 update)

In May 2025, Google announced several **significant enhancements** to ADK and related products. A **stable 1.0.0 release of the Python ADK** was released, marking the framework's readiness for production workloads ⁴. Google also introduced a **Java ADK (v0.1.0)** to bring ADK's capabilities to the Java ecosystem, enabling Java developers to build agents by adding a simple Maven dependency ⁵. A new **Agent Engine UI** provides an interface within the Google Cloud console that lets developers deploy, monitor and debug agents, list sessions, and view metrics and traces ⁶. Finally, the **Agent-to-Agent (A2A) protocol** was upgraded to version 0.2 with support for stateless interactions and standardized authentication; Google released an official A2A Python SDK and highlighted growing adoption by partners across industries ⁷. These updates underscore Google's commitment to making ADK a production-ready, multi-language platform with robust tooling and secure inter-agent communication.

Core pillars and features

Pillar/Feature	Description and citations
Multi-agent by design	ADK lets developers build modular multi-agent applications by composing specialized agents in hierarchical structures ⁸ . Agents can delegate tasks to sub-agents and coordinate complex workflows.
Rich model ecosystem	ADK can work with various LLMs. The documentation notes that ADK supports multiple models through two mechanisms: (1) specifying a model name or endpoint string for models tightly integrated with Google Cloud (e.g., Gemini), and (2) using wrapper classes such as LiteLLM for non-Google models ⁹ . Gemini is the default, but the blog notes that ADK also integrates with other providers such as Anthropic, Meta, Mistral and AI21 via LiteLLM ¹⁰ .

Pillar/Feature	Description and citations
Rich tool ecosystem	Agents can use pre-built tools (e.g., search, code execution), Google Cloud tools, third-party tools, and custom tools. Tools can include other agents (Agent-as-a-tool), enabling hierarchical compositions ¹¹ .
Flexible orchestration	Developers can define workflows using Sequential , Parallel or Loop workflow agents for predictable pipelines, or use LlmAgent for dynamic routing ¹² . This flexibility allows deterministic task pipelines or LLM-driven branching.
Built-in streaming	ADK supports bidirectional audio/video streaming for human-like conversations ¹³ . This is optional and requires Gemini models that support the Live API ¹⁴ .
Integrated developer experience	ADK provides a CLI and a web UI to develop, debug and inspect events and state step-by-step ¹⁵ .
Built-in evaluation	ADK contains evaluation hooks to assess agents' final responses and step-by-step trajectories against test cases ¹⁶ .
Easy deployment	Agents can be deployed locally, via Cloud Run, Kubernetes (GKE), or custom environments. There is support for containerization and GKE-based code execution ¹⁷ .

Model integration and authentication

The **Models & Authentication** guide explains that ADK is designed to integrate with different LLMs using two mechanisms: (1) provide a model name or endpoint directly (for Gemini or Vertex AI models), or (2) pass a wrapper object (e.g., `LiteLlm`) to the agent's `model` parameter ⁹ . This allows the use of third-party or locally hosted models. Authentication methods include API keys for Google AI Studio, user credentials or service accounts for Vertex AI, and similar credentials for other providers ¹⁸ .

2. Anatomy of an ADK agent

Agent types

1. **LlmAgent** – delegates to an LLM to decide which tools to call. It can be combined with a code executor to enable code execution.
2. **Workflow agents** – *Sequential*, *Parallel* and *Loop* agents orchestrate deterministic workflows. These are useful for predictable pipelines such as ETL or evaluation.
3. **Custom agents** – implement bespoke logic (Python/Java) for specialized behaviors.
4. **Agent-as-a-Tool** – treat another agent as a callable tool to build hierarchical systems.

Tools

ADK's built-in tools provide ready-to-use capabilities. Examples include:

- **Google Search tool** – allows agents to search the web using Google (requires Gemini models and must display search suggestions) ¹⁹ .

- **Code execution tool** – the `built_in_code_execution` tool lets the agent execute Python code when using Gemini 2 models ²⁰. For production deployments there is a **GKE Code Executor** which runs code in a sandboxed Kubernetes job ¹⁷.
- **Vertex AI RAG retrieval** and **Vertex AI Search** – tools for retrieving private documents via Google's RAG engine or searching across private data stores ²¹ ²². These tools require configuration of a RAG corpus or Vertex AI datastore.
- **BigQuery toolset** – includes methods to list datasets, obtain table information, execute SQL, and even answer natural-language questions about BigQuery tables ²³.

Developers can also **create custom function tools** when pre-built tools don't meet requirements. The function tools guide emphasizes that ADK automatically inspects a Python function's signature and docstring to build a schema for the LLM ²⁴. Required parameters must have type hints; optional parameters are those with default values ²⁵. The return type should ideally be a dictionary to provide structured context for the LLM ²⁶. Tools can pass data between calls using the `temp:` state within an invocation ²⁷. ADK also supports **long-running function tools** for tasks that take significant time and may need progress updates ²⁸.

Sessions, memory and state

Agents run inside *sessions*. ADK provides memory primitives to maintain context across turns, and a `temp:` namespace for data shared within one invocation ²⁷. For persistent memory across sessions, developers can implement custom memory stores or use built-in solutions described in the *Sessions & Memory* guide (not accessed here).

Observability and evaluation

ADK emits **events** for each tool call and agent decision, enabling detailed logging and debugging. Built-in evaluation tools measure both final responses and intermediate steps against test cases ¹⁶. Observability integrations exist for third-party tools like AgentOps, Arize AX and Weights & Biases.

3. Analytics libraries for the code execution environment

The new sub-agent will help users explore and perform analytics using Python libraries within your code execution environment. Below is a summary of the requested libraries.

Pandas

- **Purpose:** `pandas` is a **fast, powerful, flexible and easy-to-use** data analysis and manipulation tool built on top of Python ²⁹.
- **Data structures:** It offers **Series** (one-dimensional labelled arrays) and **DataFrame** (two-dimensional tables with rows and columns) ³⁰. These structures allow operations such as indexing, filtering, grouping, merging, reshaping, pivoting, handling missing data, time-series operations and reading/writing from many formats (CSV, Parquet, SQL, etc.).
- **Why important:** Pandas forms the backbone of data manipulation. It will let the analytics agent load data from your OLAP warehouse, clean it, join tables, compute summaries and pass DataFrames to other tools.

Seaborn & Matplotlib

- **Matplotlib:** A **comprehensive library for creating static, animated and interactive visualizations** in Python. Matplotlib “makes easy things easy and hard things possible” ³¹ and supports publication-quality plots, interactive figures, custom styles and exporting to many formats.
- **Seaborn:** Built on top of Matplotlib, Seaborn provides a **high-level interface** for drawing *attractive and informative statistical graphics* ³². It offers declarative functions for relational, distribution, categorical and regression plots, plus themes and colour palettes.
- **Why important:** These libraries enable visual exploration of data. The agent can automatically choose a suitable plot (e.g., scatter, histogram, box plot) based on the user’s question and call a function tool that generates the plot via Matplotlib or Seaborn.

Scikit-learn (sklearn)

- **Overview:** Scikit-learn is a **free and open-source machine-learning library** for Python ³³. It includes algorithms for **classification, regression and clustering** such as support-vector machines, random forests, gradient boosting, K-means and DBSCAN ³⁴. It is designed to interoperate with NumPy and SciPy ³³.
- **Why important:** Scikit-learn provides the building blocks for predictive models (e.g., linear regression, decision trees) and unsupervised methods (clustering, dimensionality reduction). The analytics agent can offer explanations of model choices, train models on user data via the code executor, and return metrics such as accuracy or feature importance.

Statsmodels

- **Overview:** Statsmodels is a Python module that supplies classes and functions for **estimating many statistical models**, conducting statistical tests and exploring data. It provides extensive result statistics for each estimator, and the results are tested against existing statistical packages for correctness ³⁵.
- **Typical uses:** Linear and generalized linear models, time-series models (ARIMA, SARIMA), robust regression, mixed effects models, hypothesis tests and diagnostics.
- **Why important:** Statsmodels complements Scikit-learn by offering traditional statistical modelling (with p-values, confidence intervals and hypothesis tests) and advanced time-series analysis. The analytics agent can guide users through building and interpreting these models.

Altair

- **Overview:** Vega-Altair is a **declarative visualization library** for Python with a simple, friendly API built on the Vega-Lite grammar ³⁶. Its design encourages specifying *what* to plot rather than *how* to plot it; the library handles scales, axes and marks automatically.
- **Why important:** Altair integrates well with Pandas, enabling interactive charts that can be embedded in dashboards or notebooks. It is especially useful for exploring large datasets because it produces charts that can be used in interactive front ends (via Vega-Lite).

4. Designing a natural-language analytics sub-agent

Role of the analytics agent

The goal is to create a sub-agent that acts as a **learning, brainstorming and exploration assistant** for analytics tasks. It should help users understand what analyses are possible, recommend plots and models, and answer conceptual questions without immediately writing code. When code execution is required, the agent will delegate to a separate **code execution agent** (or use ADK's built-in code executor) to run Python code with the specified libraries.

Proposed architecture

1. **Root agent** – orchestrates overall conversation and delegates tasks. It can decide whether the user's request involves analysis, coding, data retrieval or other actions. The root agent could be a `LlmAgent` with instructions to route analytical queries to the analytics sub-agent and code-heavy tasks to the code executor.
2. **Analytics sub-agent** – an `LlmAgent` specialized in analytics reasoning.
3. **Instruction:** Explain that the agent helps explore and interpret data, suggest appropriate analyses and visualizations, and call the relevant tools when needed.
4. **Tools:**
 - *Function tools for analysis tasks.* Each tool wraps a Python function that calls the required library via the code executor. Example tools:
 - `describe_dataframe(df_name: str) → Dict` – returns summary statistics and column types using pandas.
 - `visualize_relation(df_name: str, x: str, y: str, plot_type: str)` – uses Seaborn/Matplotlib to draw scatter, bar or line plots and returns a rendered image or a path.
 - `train_regression_model(df_name: str, target: str, features: List[str], model_type: str)` – trains a Scikit-learn model and returns performance metrics.
 - `perform_time_series_analysis(series_name: str, model: str)` – uses Statsmodels to fit ARIMA or other time-series models and returns summary statistics.
 - `create_altair_chart(df_name: str, x: str, y: str, chart_type: str)` – builds an Altair chart specification.
 - *RAG or search tools.* Integrate a retrieval tool (e.g., Vertex AI RAG) to answer conceptual questions by referring to documentation or internal knowledge bases.
 - *BigQuery or database tool.* Use the BigQuery toolset to fetch data directly from your OLAP warehouse ²³.
5. **Memory:** Use `temp:` state variables to pass DataFrames or analysis results between tools ²⁷. For persistent context (e.g., loaded data), implement a custom memory store.
6. **Return type:** Ensure that each function tool returns a dictionary with a `status` key and descriptive fields (e.g., `summary`, `chart_url`, `model_metrics`) to help the LLM reason ²⁶.
7. **Code execution agent** – specialized `LlmAgent` with a `code_executor` (e.g., `BuiltInCodeExecutor` or `GkeCodeExecutor`) set. Its instruction might be: "Given detailed instructions, write and execute Python code. Respond with the output and any files generated. Do

not explain the code unless asked.” This agent will receive structured instructions from the analytics sub-agent and run code accordingly. The built-in code execution tool allows Python code execution within a safe environment ²⁰. For production, you can use the GKE Code Executor to run code in a sandboxed Kubernetes job ¹⁷.

Workflow

1. **User query → root agent:** The user asks a natural-language question about data analysis.
2. **Root agent delegates:** If the question is conceptual (e.g., “What is linear regression?”) or exploratory (e.g., “How can I visualize the relationship between sales and time?”), the root agent calls the analytics sub-agent. If code must be executed, the analytics agent writes a clear instruction and calls the code execution agent via agent-as-a-tool.
3. **Analytics agent reasoning:** The analytics agent may use retrieval tools to answer general questions, suggest appropriate libraries, or call function tools to generate analysis. It ensures that the returned results are interpreted (e.g., summarizing model metrics or plot insights).
4. **Code execution:** When needed, the code execution agent runs Python code with the required libraries, returning outputs (tables, charts, metrics) to the analytics agent, which then explains them to the user.
5. **Iterative exploration:** The user can iterate, ask follow-up questions or request additional analyses. The memory and state features of ADK enable context across turns.

Implementation best practices

- **Define clear instructions and descriptions** for each agent. ADK heavily relies on `instruction` and `description` fields to inform the LLM’s behavior ³⁷. For the analytics agent, clearly state that it must decide whether to answer directly, call function tools or delegate to the code execution agent.
- **Write comprehensive docstrings** for function tools. The docstring becomes the tool’s description, which the LLM uses to determine when and how to call the tool ²⁴. Include parameter descriptions, expected input types and what the tool returns.
- **Use descriptive return dictionaries** (e.g., include `status`, `summary`, `error_message`) so the LLM can reason about outcomes ²⁶.
- **Handle errors gracefully** within tools. For example, if a DataFrame or column doesn’t exist, return an error message and status instead of raising exceptions. The LLM can then decide how to proceed.
- **Secure data access.** When querying your OLAP warehouse, use the BigQuery toolset with appropriate authentication. For privacy, restrict the analytics agent’s access to authorized datasets.
- **Evaluate and iterate.** Use ADK’s evaluation hooks to test the analytics agent with sample questions and refine its behavior ¹⁶.

5. Summary

Google’s Agent Development Kit provides a modern, flexible platform for building multi-agent AI applications. Its model-agnostic design, rich tool ecosystem and powerful orchestration primitives make it suitable for developing a **natural-language analytics assistant**. By leveraging built-in tools (code execution, RAG, BigQuery) and creating custom function tools around `pandas`, `Seaborn/Matplotlib`, `scikit-learn`, `statsmodels` and `Altair`, you can build an analytics sub-agent that helps users explore their data, answer conceptual questions, and run sophisticated analyses. The multi-agent

architecture ensures that natural-language reasoning and code execution remain decoupled yet coordinated, while ADK's evaluation and observability features facilitate iterative improvement. Following the guidelines above will help you design a robust analytics assistant tailored to your enterprise data environment.

1 Agent Development Kit

<https://google.github.io/adk-docs/>

2 3 8 10 11 12 13 15 16 37 Agent Development Kit: Making it easy to build multi-agent applications - Google Developers Blog

<https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/>

4 5 6 7 What's new with Agents: ADK, Agent Engine, and A2A Enhancements - Google Developers Blog

<https://developers.googleblog.com/en/agents-adk-agent-engine-a2a-enhancements-google-io/>

9 14 18 Models & Authentication - Agent Development Kit

<https://google.github.io/adk-docs/agents/models/>

17 19 20 21 22 23 Built-in tools - Agent Development Kit

<https://google.github.io/adk-docs/tools/built-in-tools/>

24 25 26 27 28 Overview - Agent Development Kit

<https://google.github.io/adk-docs/tools/function-tools/>

29 pandas - Python Data Analysis Library

<https://pandas.pydata.org/>

30 10 minutes to pandas — pandas 2.3.2 documentation

https://pandas.pydata.org/docs/user_guide/10min.html

31 Matplotlib — Visualization with Python

<https://matplotlib.org/>

32 seaborn: statistical data visualization — seaborn 0.13.2 documentation

<https://seaborn.pydata.org/>

33 34 scikit-learn - Wikipedia

<https://en.wikipedia.org/wiki/Scikit-learn>

35 StatsModels: Statistics in Python — statsmodels 0.6.1 documentation

<https://www.statsmodels.org/0.6.1/>

36 Vega-Altair: Declarative Visualization in Python — Vega-Altair 5.5.0 documentation

<https://altair-viz.github.io/>